

Guía de Implementación — Microservicios Krypton

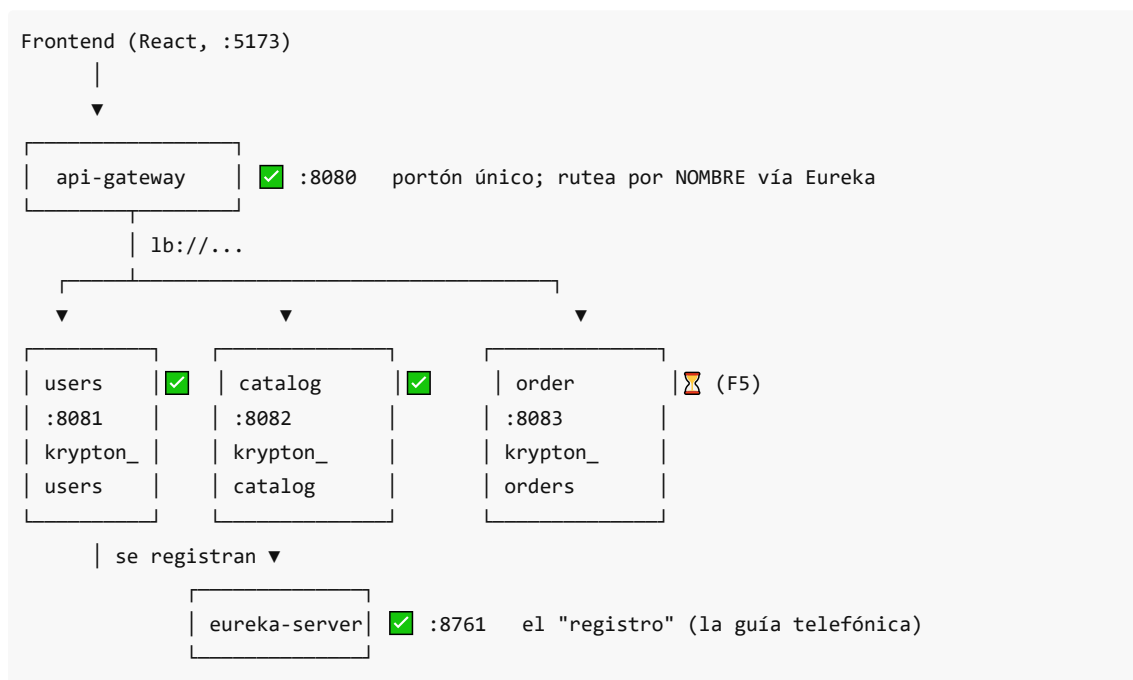
Documento vivo y DIDÁCTICO. No solo muestra el código: lo explica **línea por línea**, con el **porqué** de cada decisión y **dónde conecta** con el resto del sistema. La idea es que entiendas, no que copies. **Versión actual: hasta F4 de 12** (Eureka, users, gateway, catalog).

0. Punto de partida y objetivo

Tenemos un **monolito** Spring Boot (backend/) que funciona (411 tests verdes, frontend andando). Lo estamos partiendo en **9 microservicios** para aprender y demostrar los patrones del curso: **Eureka** (descubrimiento), **Feign** (llamadas síncronas entre servicios), **RabbitMQ** (eventos asíncronos), **Docker** y **Kubernetes**.

Regla de oro del proyecto: **no reescribimos lógica**. Reutilizamos el código del monolito, lo repartimos por servicio, y le sumamos la **capa distribuida**. El dominio (productos, usuarios, órdenes) ya está probado; lo que aprendemos acá es cómo esos pedazos **se hablan entre sí** cuando viven en procesos separados.

Arquitectura a la que apuntamos (estado actual marcado en)



Tres ideas que se repiten y conviene tener clavadas desde ya:

- Cada servicio tiene su propia base** (`krypton_users` , `krypton_catalog` , ...). Nadie lee la tabla de otro. Si necesitas un dato ajeno, lo **pedís** (Feign) o lo **escuchás** (RabbitMQ).
- Nadie hardcodea direcciones**. Los servicios se llaman **por nombre** y Eureka traduce nombre → IP:puerto.
- El token JWT es el pasaporte**. Lo emite `users` , y **todos** lo verifican con el mismo secreto, sin volver a preguntarle a `users` .

1. Decisiones de base (el andamiaje)

Monorepo + parent pom + BOM de Spring Cloud

Todos los servicios viven en `services/` como **módulos Maven** de un **parent pom**. El parent fija **una sola vez** la versión de Spring Boot y de Spring Cloud. Este es el *gotcha* #1 del stack: **Spring Cloud va atada a una versión específica de Spring Boot**; si las mezclás mal, no arranca.

`services/pom.xml` (lo esencial), comentado:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.3.5</version>          <!-- 1) hereda gestión de deps de Spring
Boot 3.3.5 -->
</parent>

<groupId>pe.com.krypton</groupId>
<artifactId>krypton-services</artifactId>      <!-- 2) el "paraguas" de todos los
servicios -->
<version>1.0.0</version>
<packaging>pom</packaging>                  <!-- 3) NO genera jar: solo orquesta
módulos -->

<properties>
  <java.version>17</java.version>
  <spring-cloud.version>2023.0.3</spring-cloud.version> <!-- 4) versión de Cloud
compatible con Boot 3.3.x -->
</properties>

<modules>                                <!-- 5) los servicios que compila el
parent -->
  <module>eureka-server</module>
  <module>users-service</module>
  <module>api-gateway</module>
  <module>catalog-service</module>
</modules>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>          <!-- 6) el BOM: decide la versión de
CADA starter de cloud -->
    </dependency>
  </dependencies>
</dependencyManagement>
```

Qué hace cada parte numerada:

1. **<parent> Spring Boot**: heredamos el `dependencyManagement` de Boot — por eso en los hijos las deps de Spring van **sin** `<version>`. 2–3. **packaging=pom**: este módulo no es una app, es un **agregador**. Su único trabajo es compilar a los `<modules>` en orden y compartirles la config.

2. `spring-cloud.version` : una property para no repetir el número.
3. **El BOM** (`<scope>import</scope>`): un "Bill of Materials". Importa una tabla gigante de versiones de TODO el ecosistema Spring Cloud. Gracias a esto, en cada servicio el starter de Eureka/Gateway va **sin** `<version>` y nunca chocan entre sí.

Por qué importa: mezclar versiones sueltas de Spring Cloud es la causa #1 de errores crípticos al arrancar. El BOM lo elimina de raíz: una sola fuente de verdad para las versiones.

Convenciones que se repiten en TODOS los servicios

- Clase main en `pe.com.krypton.XxxApplication` (en el **paquete base**, para que el escaneo de componentes abarque todo el código carvado, que conserva paquetes `pe.com.krypton.*`).
- `application.yml` con: `server.port` , `spring.application.name` (el nombre en Eureka), `datasource` a **su propio schema**, y `eureka.client.service-url.defaultZone` .
- Una base por servicio (`krypton_users` , `krypton_catalog` , ...) en la misma instancia MySQL.

2. F1 — eureka-server (service discovery)

Concepto: ¿por qué un "registro"?

En un monolito, todo está en el mismo proceso: llamar a otra clase es un método. En microservicios, "el otro" es **otro proceso, en otra IP, en otro puerto**, que además puede **escalar** (3 copias), **caerse** y **revivir** con otra dirección. Si `http://192.168.0.5:8082` , el día que esa instancia cambie, todo se rompe.

Eureka es una guía telefónica viva. Cada servicio, al arrancar, **se anota** ("soy `catalog-service` , estoy en tal IP:puerto"). Los demás, cuando necesitan a `catalog-service` , le **preguntan a Eureka por el nombre** y reciben la dirección actual. Nombres en vez de direcciones.

Código — la app

`eureka-server/pom.xml` lleva **una sola** dependencia de cloud:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>  <!-- el SERVIDOR
de registro -->
</dependency>
```

`EurekaServerApplication.java` :

```
@SpringBootApplication          // 1) app Spring Boot normal (autoconfig + component-scan)
@EnableEurekaServer              // 2) ← LA línea clave: convierte esta app en el registro
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);  // 3) arranca el
contexto y el server web
    }
}
```

1. `@SpringBootApplication` : el combo de siempre (`@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`). Sin esto no hay app.

2. **@EnableEurekaServer** : la magia. Levanta el servidor de registro y su panel web. **Es lo único** que diferencia a esta app de un Spring Boot vacío. (Ojo: el SERVER usa **@EnableEurekaServer** ; los CLIENTES usan **@EnableDiscoveryClient** , que viene en F2.)
3. **main** : arranque estándar de Spring Boot.

application.yml — acá está la sutileza:

```
server:
  port: 8761 # 1) puerto ESTÁNDAR de Eureka (todos lo asumen)

spring:
  application:
    name: eureka-server # 2) su nombre

eureka:
  client:
    register-with-eureka: false # 3) el server NO se registra a sí mismo
    fetch-registry: false # 4) ni descarga el registro de otros (él ES el registro)
  instance:
    hostname: localhost # 5) cómo se anuncia a sí mismo (en local, localhost)
```

1. **8761** : el puerto canónico de Eureka. Por convención, todos los clientes apuntan acá. 3–4. **register-with-eureka: false / fetch-registry: false** : el detalle que confunde a todos. El servidor de Eureka **también trae un cliente** adentro (porque Eureka puede correr en clúster, replicándose entre varios servers). Como nosotros tenemos **uno solo**, le decimos: "no intentes registrarte ni buscar a otros, vos sos el único registro". Sin estas dos líneas, el server arranca tirando errores de conexión consigo mismo.
2. **hostname: localhost** : en desarrollo, cómo se identifica. En Docker/K8s esto cambia.

Dónde conecta

Es el punto al que **todos** apuntan con `defaultZone: http://localhost:8761/eureka` . Sin él, nadie se encuentra por nombre. Panel: `http://localhost:8761` (lo ves en el navegador con la lista de servicios UP).

Verificado

Arranca en ~6s. El registro arranca **vacío** (`"application":[]`) — correcto, todavía nadie se anotó. En cuanto levantan users y catalog, aparecen ahí como `USERS-SERVICE` y `CATALOG-SERVICE` .

3. JWT a fondo — cómo funciona DE VERDAD

Antes de tocar `users` (que **emite** el token) y `catalog` (que lo **valida**), paremos la pelota y entendamos qué es un JWT por dentro. Si esto no te queda CLARO, la seguridad de todos los servicios va a ser una caja negra. Y nosotros no trabajamos con cajas negras.

El problema que resuelve

HTTP es **sin estado** (stateless): cada request es independiente, el servidor no "recuerda" el anterior. Entonces, ¿cómo sabe el servidor que vos ya te logueaste? Dos enfoques:

- **Sesión clásica (stateful)**: el server guarda en memoria/DB "el usuario 42 está logueado" y te manda una cookie con un id de sesión. En CADA request busca esa sesión. Problema en microservicios: si tenés 5

servicios, ¿dónde vive la sesión? Tendrías que compartir un store de sesiones entre todos. Acopla y escala mal.

- **Token autocontenido (stateless): JWT.** El server te da un papel **firmado** que dice quién sos. Vos lo guardás y lo mandás en cada request. El server **no guarda nada**: lee el papel, verifica la firma, y listo. **Esto** es lo que escala a N servicios: cada uno valida el papel por su cuenta.

Anatomía de un JWT: tres partes separadas por puntos

Un JWT es un string que se ve así (cortado para que entre):

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ9LnBlIiwicm9sZSI6IkpFETU10IiwiaWF0Ijo... .
e3dsUMb432QMLrt1QfWVbvq2_V4TG2GTocPATkrmpk...
|----- HEADER -----| |----- PAYLOAD -----|
|----- SIGNATURE -----|
```

Tres partes: **header.payload.signature**. Las dos primeras son **JSON codificado en Base64URL** (NO encriptado — cualquiera lo puede leer). La tercera es la **firma**.

Base64URL ≠ encriptación. Es solo una forma de representar bytes con caracteres seguros para URLs. Si agarrás el header `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ9LnBlIiwicm9sZSI6IkpFETU10IiwiaWF0Ijo...` y lo decodificás (lo hicimos hoy con `base64 -d`), sale:

```
{ "alg": "HS384" }
```

Y el payload decodificado de NUESTRO token real es:

```
{ "sub": "admin@krypton.pe", "role": "ADMIN", "iat": 1782490665, "exp": 1782577065 }
```

- **sub** (subject): a quién pertenece el token → el email.
- **role**: un **claim** (afirmación) nuestro, custom → el rol. Esto es lo que lee `catalog` para decidir si sos ADMIN, **sin tocar la DB**.
- **iat** (issued at) y **exp** (expiration): timestamps Unix. El token vence en `exp`.

🔑 **Punto que la gente no entiende:** el payload **NO está oculto**. Cualquiera que tenga el token puede leer tu email y tu rol. JWT **no da confidencialidad**, da **integridad**: garantiza que nadie **modificó** el contenido. Por eso **NUNCA** metas un password o un dato secreto en el payload.

La firma: por qué NO se puede falsificar

Acá está el corazón. La firma se calcula así:

```
firma = HMAC-SHA384( base64url(header) + "." + base64url(payload) , SECRETO )
```

`HMAC` es una función que toma **dos cosas** — el contenido y una **clave secreta** — y escupe un hash. Propiedades clave:

- **Determinista**: mismo contenido + mismo secreto = misma firma, siempre.
- **Sensible**: si cambiás UN carácter del payload, la firma cambia por completo.
- **Irreversible sin el secreto**: sin conocer el SECRETO, no podés calcular una firma válida.

Entonces, ¿qué pasa si un atacante intercepta tu token y cambia `"role": "ADMIN"` ... perdón, `"role": "CLIENTE"` por `"role": "ADMIN"` ?

1. Modifica el payload → el JSON ahora dice ADMIN.

2. Pero la firma vieja ya **no corresponde** a ese payload nuevo.
3. Para que cuele, necesitaría **recalcular la firma...** y para eso necesita **el SECRETO**.
4. El secreto vive **solo en los servidores**, nunca viaja. → **No puede**. El servidor recalcula la firma del payload recibido, ve que no coincide, y rechaza el token. 401.

Lo verificamos hoy con openssl: tomamos el `header.payload` del token real, calculamos `HMAC-SHA384` con el secreto `dev-only-...`, y la firma nos dio **idéntica** a la del token. Esa es exactamente la cuenta que hace el servidor en cada request.

Por qué nuestro token es HS384 (y no HS256)

Detalle fino y real de NUESTRO código. En `JwtService` la clave se arma así:

```
this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
```

`Keys.hmacShaKeyFor` **elige el algoritmo según el LARGO del secreto:**

Largo del secreto	Algoritmo que elige
≥ 256 bits (32 bytes)	HS256
≥ 384 bits (48 bytes)	HS384
≥ 512 bits (64 bytes)	HS512

Nuestro secreto `dev-only-change-me-krypton-secret-key-1234567890` tiene **exactamente 48 bytes = 384 bits**, así que jjwt usa **HS384** (lo confirma el header `{"alg": "HS384"}`). Curiosidad: el javadoc de la clase dice "HS256" — **es impreciso**, el runtime manda. Moraleja didáctica: el algoritmo no lo decide un comentario, lo decide el **largo de la clave**.

Stateless: el secreto compartido es el "contrato"

```
users-service —firma con SECRETO→ TOKEN —verifica con el MISMO SECRETO→ catalog-  
service  
    (emisor)  
    (validador)
```

`users` es el **único** que puede emitir tokens (es dueño de la tabla `users` y conoce los passwords). Pero CUALQUIER servicio que tenga **el mismo secreto** puede **verificar** un token sin preguntarle a `users`. Por eso `catalog` valida en milisegundos, offline, sin llamadas de red. El secreto compartido es **el contrato de seguridad** entre servicios. Cambialo en uno y rompés a todos: esa es justo la prueba de que el sistema funciona.

Con esto en la cabeza, las dos próximas fases (users que EMITE, catalog que VALIDA) se leen solas.

4. F2 — users-service (el emisor del JWT + primer cliente de Eureka)

Concepto

Es el primer servicio que **se registra** en Eureka. Convierte un Spring Boot normal en cliente de Eureka con **3 cosas**: la dependencia, el `application.name` y el `defaultZone`. Carva del monolito la lógica de **usuarios + autenticación**, con su base `krypton_users`. Y es el **único que emite** el JWT.

La app (cliente de Eureka)

```
@SpringBootApplication
@EnableDiscoveryClient           // ← convierte la app en CLIENTE de Eureka (se registra al
                                arrancar)
public class UsersServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UsersServiceApplication.class, args);
    }
}
```

La **única** diferencia con el eureka-server es la anotación: acá `@EnableDiscoveryClient` (me anoto y busco a otros), allá `@EnableEurekaServer` (yo SOY el registro).

`application.yml` — las "3 cosas" + su base + el JWT:

```
server:
  port: 8081
spring:
  application:
    name: users-service           # 1) nombre con el que lo encuentran
    los demás
  datasource:
    url: jdbc:mysql://localhost:3307/krypton_users # 2) SU base, nadie más la toca
    username: krypton
    password: krypton
  jpa:
    hibernate:
      ddl-auto: validate         # 3) Flyway manda el schema;
    Hibernate solo valida
      open-in-view: false
  flyway:
    enabled: true
  app:
    jwt:
      secret: ${JWT_SECRET:dev-only-change-me-krypton-secret-key-1234567890} # 4) el secreto
      (override por env en prod)
      expiration: 86400000       # 5) 24h en milisegundos
  eureka:
    client:
      service-url:
        defaultZone: http://localhost:8761/eureka # 6) dónde está el registro
```

2. `/krypton_users` : su base propia. Aislamiento de datos = cada servicio dueño de lo suyo.
3. `ddl-auto: validate` : Hibernate NO crea ni modifica tablas; solo verifica que el schema (que arma Flyway) matchee las entidades. Si no coincide, no arranca. Disciplina pura.
4. `${JWT_SECRET:default}` : lee la env var `JWT_SECRET` ; si no existe, usa el default de dev. En prod inyectás un secreto real por entorno. Este MISMO valor tiene que estar en catalog.
5. `expiration: 86400000` : 86.400.000 ms = 24h. Cuánto vive el token.

6. **defaultZone** : la dirección del registro. Las "3 cosas" para ser cliente son los puntos 1 (nombre), 6 (registro) y la dependencia eureka-client del pom.

El login: dónde NACE el token

AuthController.java — la puerta de entrada HTTP:

```
@RestController
@RequestMapping("/api/auth")           // 1) todas las rutas cuelgan de /api/auth
public class AuthController {

    private final AuthService authService; // 2) depende de la INTERFAZ (no del impl) →
    // testeable

    public AuthController(AuthService authService) { // 3) inyección por constructor
        this.authService = authService;
    }

    @PostMapping("/login")              // 4) POST /api/auth/login
    public AuthResponse login(@Valid @RequestBody LoginRequest request) { // 5) valida y
    // deserializa el body
        return authService.login(request); // 6) delega TODA la lógica al service
    }
}
```

2. **Depende de AuthService (interfaz)**, no de la implementación → en los tests mockeás la interfaz. Regla de oro de la arquitectura por capas.
3. **@Valid @RequestBody** : Spring deserializa el JSON a `LoginRequest` y corre las validaciones (email no vacío, etc.). Si fallan → 400 automático, ni entra al método.
4. **El controller NO tiene lógica**: delega. Su trabajo es HTTP (rutas, status, serialización).

AuthServiceImpl.login(...) — la lógica real:

```
@Override
@Transactional(readOnly = true)        // 1) solo lee; readOnly optimiza
public AuthResponse login(LoginRequest request) {
    Usuario user = userRepository.findByEmail(request.email()) // 2) busca por
    // email

    .orElseThrow(() -> new InvalidCredentialsException("Credenciales inválidas"));
    if (!passwordEncoder.matches(request.password(), user.getPassword())) { // 3) compara
    // hash, NUNCA texto plano
        throw new InvalidCredentialsException("Credenciales inválidas");
    }
    if (!user.isActive()) { // 4) usuario dado
    // de baja no entra
        throw new InvalidCredentialsException("Credenciales inválidas");
    }
    return new AuthResponse(jwtService.generateToken(user), // 5) ← acá NACE el
    // token
        "Bearer", expirationMs / 1000); // 6) tipo +
```



```
segundos de vida
}
```

2. `findByEmail` devuelve `Optional` ; si no existe, `orElseThrow` .
3. `passwordEncoder.matches(plano, hash)` : BCrypt. En la DB el password está **hasheado**; comparás el texto que llegó contra el hash. Nunca se guarda ni compara en plano. 4-. **Mismo error para los 3 casos** (email inexistente / password mal / inactivo): a propósito, no le revelamos a un atacante **qué** emails existen. Detalle de seguridad real.
4. `jwtService.generateToken(user)` : el momento exacto en que se crea el JWT. Le pasamos el usuario entero porque adentro necesita su email (subject) y su rol (claim).

`JwtService.generateToken(...)` — la fábrica del token:

```
public String generateToken(Usuario user) {
    Date now = new Date();
    return Jwts.builder()
        .subject(user.getEmail()) // 1) sub = email → el
"sub" del payload
        .claim("role", user.getRole().name()) // 2) claim custom: el rol
viaja DENTRO del token
        .issuedAt(now) // 3) iat = ahora
        .expiration(new Date(now.getTime() + expirationMs)) // 4) exp = ahora + 24h
        .signWith(key) // 5) FIRMA con el
secreto (HMAC) → la 3ra parte
        .compact(); // 6) arma
"header.payload.signature" y lo serializa
}
```

1. **subject** → el `sub` del payload (el dueño del token).
2. `claim("role", ...)` : metemos el rol COMO DATO en el token. `user.getRole().name()` convierte el enum `Rol.ADMIN` en el String `"ADMIN"` . Esto es lo que después `catalog` lee para autorizar **sin tocar la DB**. ← clave del stateless.
3. **exp** : `now + expirationMs` . Pasado ese instante, cualquier validador lo rechaza.
4. `signWith(key)` : calcula la firma HMAC del `header.payload` con el secreto. Sin especificar algoritmo: `jjwt` lo deduce del largo de la clave (384 bits → HS384, ver §3).
5. `compact()` : pega las 3 partes con puntos y devuelve el String final.

La validación en users: contra la DB (¡distinto a catalog!)

`users` tiene la tabla `users` , así que se da el lujo de **revalidar contra la DB** en cada request. ¿Para qué? Para que una **baja lógica tenga efecto inmediato** aunque el token siga vigente.

`JwtAuthenticationFilter` (`users`) — corre **una vez por request**:

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain chain) {
    String header = request.getHeader("Authorization"); // 1) lee el header
Authorization
    if (header != null && header.startsWith("Bearer ") // 2) ¿viene "Bearer
<token>"?
        && jwtService.isValid(header.substring(7))) { // 3) ¿firma OK y no
```

```

expirado?
    authenticate(header.substring(7), request); // 4) si sí,
autentica
    }
    chain.doFilter(request, response); // 5) sigue la
cadena (haya o no auth)
}

private void authenticate(String token, HttpServletRequest request) {
    UserDetails user =
userDetailsService.loadUserByUsername(jwtService.extractEmail(token)); // 6) ← LEE LA DB
    if (user.isEnabled()) { // 7) ¿sigue activo?
(baja inmediata)
        var auth = new UsernamePasswordAuthenticationToken(user, null,
user.getAuthorities()); // 8) identidad + roles
        auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(auth); // 9) "este request
está autenticado"
    }
}

```

3. **isValid** : verifica firma + expiración (parsea el token con el secreto; si algo falla, tira excepción que se atrapa y devuelve `false`).
4. **chain.doFilter** **SIEMPRE se ejecuta**: el filtro no rechaza; solo **autentica si puede**. Quien rechaza (401/403) es la capa de autorización, más adelante en la cadena, si no hay identidad.
5. **loadUserByUsername** → **toca la DB**. Esta es **LA diferencia con catalog**. `users` vuelve a buscar el usuario para confirmar que existe y sigue activo.
6. **isEnabled()** (= `active` en la entidad): si lo diste de baja, NO se autentica, aunque su token sea válido. Baja inmediata sin esperar a que venza el token.
7. **SecurityContextHolder** : el "casillero" por-request donde Spring Security guarda quién sos.

`CustomUserDetailsService.loadUserByUsername(...)` — traduce tu `Usuario` al modelo de Spring:

```

public UserDetails loadUserByUsername(String email) {
    Usuario user = userRepository.findByEmail(email)
        .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado: " +
email));
    return User.builder()
        .username(user.getEmail())
        .password(user.getPassword())
        .authorities(new SimpleGrantedAuthority("ROLE_" + user.getRole().name())) // ←
prefijo ROLE_ obligatorio
        .disabled(!user.isActive())
        .build();
}

```

El detalle que rompe a todos: `"ROLE_" + ...`. Spring Security, cuando usás `hasRole("ADMIN")`, internamente busca la authority `ROLE_ADMIN`. Si no ponés el prefijo `ROLE_`, `hasRole` nunca matchea y te comés 403 eternos. El prefijo es **convención obligatoria**.

`SecurityConfig (users)` — define qué es público y qué no:

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .cors(Customizer.withDefaults()) // 1)
    habilita CORS (config aparte)
        .csrf(csrf -> csrf.disable()) // 2) sin
    CSRF: somos API stateless
        .sessionManagement(sm -> sm.sessionCreationPolicy(STATELESS)) // 3) NUNCA
    crear sesión HTTP
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll() // 4)
    login/registro: público
            .requestMatchers("/api/admin/**").hasRole("ADMIN") // 5)
    gestión: solo ADMIN
            .anyRequest().authenticated() // 6) el
    resto: logueado
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(authEntryPoint) // 7) sin
    token → 401 JSON
                .accessDeniedHandler(accessDeniedHandler)) // 8) sin
    permiso → 403 JSON
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class); // 9)
    nuestro filtro ANTES del de Spring
        return http.build();
}

```

2. **CSRF disabled**: CSRF protege formularios con sesión/cookies. Nosotros usamos un token en un header, no cookies de sesión → no aplica.
3. **STATELESS** : la línea filosófica. Spring Security **no crea ni usa HttpSession** . Cada request se autentica de cero con su token. Coherente con el JWT. 4–6. **Las reglas se evalúan EN ORDEN**, de la más específica a la más general. `/api/auth/**` público (para poder loguearte sin estar logueado), `/api/admin/**` solo ADMIN, todo lo demás autenticado.
4. **addFilterBefore** : enchufamos nuestro `JwtAuthenticationFilter` **antes** del filtro de login por usuario/password de Spring. Así, cuando llega un request, primero intentamos autenticar por token.

Dónde conecta

Al arrancar, se registra en Eureka como `USERS-SERVICE` . El token que emite acá lo **validan los demás** con el mismo secreto (F4). El gateway lo expone en `/api/auth/**` y `/api/admin/users/**` .

Verificado

`USERS-SERVICE` aparece UP en Eureka. `POST :8080/api/auth/login` con `admin@krypton.pe / Admin123!` devuelve un token Bearer. Y `GET /api/admin/users` con ese token → **200** (valida contra la DB).

5. F3 — api-gateway (portón único)

Concepto

Con varios servicios en puertos distintos, el frontend no debería conocer cada uno (¿y si mañana catalog cambia de puerto?). El **gateway** es **una sola puerta** (`:8080` , donde ya apunta el front) que **rutea por nombre** vía Eureka y

centraliza el **CORS**. El front sigue hablando con `:8080` como si fuera el viejo monolito: **cero cambios en el front**.

Por qué el gateway es REACTIVO (no lleva spring-web)

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>      <!-- gateway: stack
REACTIVO (WebFlux) -->
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>  <!-- para resolver
lb://nombre -->
</dependency>
```

Gotcha real: Spring Cloud Gateway corre sobre **WebFlux** (reactivo, no bloqueante). Si por inercia agregás `spring-boot-starter-web` (el servlet clásico), **chocan los dos stacks** y la app no arranca. El gateway NO lleva `-web`. Tiene sentido: un portón que solo reenvía miles de requests rinde mejor sin bloquear un hilo por cada uno.

ApiGatewayApplication.java :

```
@SpringBootApplication
@EnableDiscoveryClient          // ← también es cliente de Eureka: necesita preguntar
"¿dónde está X?"
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }
}
```

application.yml — el corazón del ruteo:

```
server:
  port: 8080                # 1) donde apunta el front → no cambia nada del front
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      default-filters:
        - DedupeResponseHeader=Access-Control-Allow-Origin Access-Control-Allow-Credentials
        RETAIN_UNIQUE # 2)
      globalcors:
        cors-configurations:
          '[/**]':           # 3) CORS para TODAS las rutas
            allowedOrigins: ["http://localhost:5173"] # solo el front de dev
            allowedMethods: [GET, POST, PUT, PATCH, DELETE, OPTIONS]
            allowedHeaders: [Authorization, Content-Type]
            exposedHeaders: [Content-Disposition]      # para que el front lea nombres de
archivo
```

```

        allowCredentials: true
    routes:
        # 4) la tabla de ruteo
        - id: users-auth
          uri: lb://users-service
          # 5) lb:// = resolver por Eureka + balancear
          predicates: [ "Path=/api/auth/**" ]
          # 6) si el path matchea → mandá acá
        - id: catalog-products
          uri: lb://catalog-service
          predicates: [ "Path=/api/products/**" ]
        - id: catalog-admin-products
          uri: lb://catalog-service
          predicates: [ "Path=/api/admin/products/**" ]
        # ... (categories, uploads, admin-users, etc.)
    eureka:
      client:
        service-url:
          defaultZone: http://localhost:8761/eureka
          # 7) para resolver los lb://

```

2. **DedupeResponseHeader** : gotcha clásico de CORS detrás de gateway. Tanto el gateway COMO el servicio destino pueden agregar `Access-Control-Allow-Origin` . Resultado: el header llega **duplicado** y el navegador lo rechaza. Este filtro deja **uno solo** (`RETAIN_UNIQUE`).
3. **globalcors** : el CORS se define **una vez acá**, no en cada servicio. El navegador solo habla con `:8080` , así que solo el gateway necesita política CORS.
4. **uri: lb://users-service** : el corazón. **lb://** = *load balancer*. NO es una URL fija: el gateway le pregunta a Eureka "¿dónde está `users-service` ?", obtiene la dirección actual, y si hubiera varias instancias, **reparte** (balancea). Nombre, no IP.
5. **predicates: Path=...** : la condición de match. "Si el path empieza con `/api/auth/` , esta ruta aplica." Se evalúan en orden hasta el primer match.

Gotcha operativo (nos pasó): esta config va **dentro del jar**. Si agregás una ruta, hay que **recompilar y reiniciar** el gateway. Y en Windows, el jar está **bloqueado** mientras corre → primero **parás** el proceso, después `mvn package` , después levantás. Si no, el build falla porque no puede sobrescribir el jar.

Dónde conecta

El front pega a `:8080` → el gateway mira el path → elige la ruta → resuelve el nombre por Eureka → reenvía. El `Authorization: Bearer` que mande el front **viaja tal cual** al servicio destino (lo comprobamos: `GET /api/admin/users` por el gateway con token = 200).

Verificado

`POST :8080/api/auth/login` → token (gateway → users). `GET :8080/api/products` → catálogo (gateway → catalog). Preflight CORS responde con **un solo** `Access-Control-Allow-Origin` .

6. F4 — catalog-service (validar un JWT que NO emitiste)

Concepto (el más importante hasta acá)

catalog **no tiene la tabla** `users` . No puede revalidar contra la DB como hace users. Y acá brilla el JWT: **es autocontenido**. catalog solo **verifica la firma** con el secreto compartido y **lee el rol del propio token**. Validación **stateless pura**: sin DB, sin sesión, sin llamar a users. Esto es lo que aprendiste en §3, ahora en código.

El JwtService de catalog: recortado a SOLO validar

```
@Service
public class JwtService {
    private final SecretKey key;
    public JwtService(@Value("${app.jwt.secret}") String secret) {           // 1) SOLO el
secreto (no expiration)
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8)); // 2) misma
clave que users (mismo secreto)
    }
    public boolean isValid(String token) {                                   // 3) ¿firma
OK y no expirado?
        try { parse(token); return true; }
        catch (JwtException | IllegalArgumentException e) { return false; } // 4)
cualquier falla → inválido
    }
    public String extractEmail(String token) { return
parse(token).getPayload().getSubject(); } // 5) lee el sub
    public String extractRole(String token) { return parse(token).getPayload().get("role",
String.class); } // 6) lee el claim
    private Jws<Claims> parse(String token) {
        return Jwts.parser().verifyWith(key).build().parseSignedClaims(token); // 7)
verifica firma y parsea
    }
}
```

1. **Constructor SOLO con secret** : catalog **no emite** tokens, así que no necesita `expiration` ni la entidad `Usuario` . Es la versión "lectora" del `JwtService` de `users`. Misma derivación de clave → **misma firma** → puede validar lo que `users` firmó. 3-4. **isValid** : intenta parsear; si la firma no matchea o expiró, `parse` tira excepción, la atrapamos y devolvemos `false` . Sin ruido.
2. **extractRole** : lee el claim `"role"` del payload. **Esto reemplaza el lookup a la DB**. El rol viene firmado dentro del token; confiar en él es seguro **porque la firma garantiza que nadie lo tocó** (todo §3 era para poder confiar en esta línea).
3. **verifyWith(key)** : el verificador. Recalcula la firma del `header.payload` con la clave y la compara con la firma del token. Si no coinciden → excepción → inválido.

El filtro de catalog: identidad desde los CLAIMS (sin DB)

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain chain) {
    String header = request.getHeader("Authorization");
    if (header != null && header.startsWith("Bearer ")) {
        String token = header.substring(7);
        if (jwtService.isValid(token)) {                                     // 1)
firma + expiración
            var authorities = List.of(new SimpleGrantedAuthority("ROLE_" +
jwtService.extractRole(token))); // 2) rol DEL TOKEN
            var auth = new
UsernamePasswordAuthenticationToken(jwtService.extractEmail(token), null, authorities); //
```

```

3)
        auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(auth);           // 4)
autenticado, sin tocar la DB
    }
}
chain.doFilter(request, response);
}

```

Compará con el filtro de users (§4): users hace `loadUserByUsername` (DB); catalog construye la identidad **directamente de los claims**.

2. **"ROLE_" + extractRole(token)** : arma la authority desde el rol que viene en el token. Cero consultas. Esta es la esencia del microservicio stateless: **confío en el token firmado**.
3. El **"principal" es el email** (String), no un `Usuario` cargado — catalog ni siquiera tiene esa entidad de usuario.
4. **Autenticado sin DB**. Latencia mínima, sin acople a users. El precio: si das de baja a un user en `users`, su token **sigue siendo válido en catalog hasta que expire** (no hay revalidación). Es el trade-off consciente de stateless. (Se resuelve con tokens cortos o una lista de revocación — fuera de alcance del curso.)

`SecurityConfig` (catalog) — casi igual a users, **pero sin** `PasswordEncoder` ni `AuthenticationManager` (catalog no autentica a nadie, solo verifica tokens):

```

.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()           // 1) ver
catálogo: público
    .requestMatchers(HttpMethod.GET, "/api/categories/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/uploads/**").permitAll()           // ver
imágenes: público
    .requestMatchers("/api/admin/**").hasRole("ADMIN")                       // 2) gestión:
solo ADMIN
    .anyRequest().authenticated())

```

1. **GET de catálogo público**: cualquiera puede VER productos sin loguearse (es un e-commerce). Ojo: solo **GET**; crear/editar/borrar caen en `/api/admin/**`.
2. `/api/admin/**` **solo ADMIN**: y acá el rol sale **del token**, validado statelessly.

`application.yml` — el secreto, que **DEBE** ser idéntico al de users:

```

app:
  jwt:
    secret: ${JWT_SECRET:dev-only-change-me-krypton-secret-key-1234567890} # = al de users
(mismo secreto = misma firma)

```

Si este valor difiere aunque sea **un byte** del de users, catalog rechaza todos los tokens (la firma no matchearía). Lo verificamos hoy comparando los dos secretos byte a byte: idénticos. **Ese es el contrato**.

Adaptación de datos: FK entre servicios

La tabla `stock_movement` tenía una FK `created_by` → `users`. Como `users` vive en **otro servicio con otra base**, no se hace FK cruzada: el campo pasa a ser un simple `Long createdBy` (sin relación JPA) y la migración

Flyway crea la columna **sin constraint**. Regla general: **una FK que cruza el límite de un servicio se degrada a un id suelto** (Long). La integridad referencial entre servicios se cuida en código/eventos, no con constraints de la DB.

Verificado (con el gotcha que cazamos hoy)

- GET :8080/api/products → 10 productos del seed (público). ✓
- POST :8080/api/admin/products **sin** token → **401** (rechaza). ✓
- POST :8080/api/admin/products **con** token admin → **400** de validación. ✓ ← el 400 (y no 401) prueba que el token **se aceptó**: pasó seguridad y llegó al controller, donde @Valid rechazó el body vacío.

 **Gotcha real que diagnosticamos:** GET /api/admin/products con token válido da **401**, no 405, aunque el endpoint solo exista para POST. ¿Por qué? El método no mapeado lanza 405 → Spring hace un **forward interno a /error** → la cadena de seguridad se re-evalúa sobre /error, pero el filtro (un OncePerRequestFilter) **no corre en error-dispatch** por default → /error queda sin autenticación → 401. Moraleja: **para probar que un servicio acepta el JWT, usá el método REAL del endpoint** (POST/PUT/DELETE), no GET sobre rutas POST-only.

7. Cómo conecta TODO (hasta la F4)

1. El front (o curl) le pega al GATEWAY en :8080 con Authorization: Bearer <token>.
2. El gateway mira el PATH y elige el servicio (ruta lb://nombre).
3. Le pregunta a EUREKA dónde está ese servicio (por nombre) y reenvía.
4. El servicio destino valida el JWT:
 - users-service → contra su DB (es dueño de los usuarios; baja inmediata).
 - los demás → STATELESS: verifica la firma con el SECRETO COMPARTIDO + lee el role del token.
5. Cada servicio trabaja contra SU PROPIA base (krypton_users / krypton_catalog / ...).

Los tres "pegamentos" del sistema, hasta acá:

- **Eureka** (nombres ↔ direcciones) — nadie hardcodea hosts.
- **El gateway** (una puerta, rutea por nombre, CORS).
- **El secreto JWT compartido** (el contrato: users firma, todos verifican).

Lo que **todavía no** aparece: **Feign** (un servicio llamando a otro — llega en F5 con el checkout) y **RabbitMQ** (eventos asíncronos — F6).

8. Cómo levantar lo implementado

Los servicios corren como **jars**; al cerrar la terminal/sesión, se caen. Para relevantarlos:

```
docker compose up -d                                # MySQL (desde la raíz del
repo) → :3307
cd services
# Orden: PRIMERO eureka; el resto puede ir en paralelo. Cada uno en su terminal o en
background:
java -jar eureka-server/target/eureka-server-1.0.0.jar      # :8761 (esperar a que
responda)
java -jar users-service/target/users-service-1.0.0.jar      # :8081
java -jar catalog-service/target/catalog-service-1.0.0.jar  # :8082
java -jar api-gateway/target/api-gateway-1.0.0.jar          # :8080
```


Compilar un servicio (en Windows, pararlo antes si está corriendo — jar bloqueado): `cd services && mvn -q -pl <servicio> -am package -DskipTests` .

Verificación rápida end-to-end:

```
curl http://localhost:8761/eureka/apps # ¿quién está registrado?
TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@krypton.pe","password":"Admin123!"}' | grep -o '"token": "[^"]*"' | cut
-d'' -f4)
curl http://localhost:8080/api/products # catálogo por el gateway
curl -X POST http://localhost:8080/api/admin/products \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{}' # → 400 =
token aceptado
```

Login de prueba: `admin@krypton.pe` / `Admin123!` .

9. Próximos pasos (se van escribiendo acá)

- **F5** — order-service: carrito + pedidos + **checkout distribuido** (Feign a catalog para descontar stock + **saga/compensación**). El núcleo conceptual: la primera vez que un servicio **llama** a otro de forma síncrona y hay que manejar el "¿y si falla a la mitad?".
- **F6** — notification-service + RabbitMQ (eventos asíncronos).
- **F7** — payment-service. **F8** — review-service. **F9** — promo-service.
- **F10** — Dockerizar todo. **F11** — Frontend de los servicios nuevos. **F12** — Kubernetes.

(Esta guía se actualiza al cerrar cada fase, siempre con el mismo nivel de detalle.)